

ANSWERS TO REVIEW QUESTIONS

MIDTERM 11/05/2013

COMPUTER & COMMUNICATION NETWORKS
UC IRVINE - FALL 2013

Notes :

- 1) Key concepts are the most important things and must be included when answering the review questions.
- 2) Obviously, do not follow the format of the answers here in your final. Coherent, reasoned answers are preferred. "Out-of-thin-air" (correct) answers will not receive full credit. Your final decides your comprehensive pass/no pass, show that you understand the materials, not remembering.

Review questions 1

R1.a (3Points) Concisely describe the client-server and the peer-to-peer application architectures and highlight their differences.

Client-server:

There is an **always-on host** (server)
Clients **don't communicate with each other**
Server has **fixed - well-known address** (its IP)
Clients can **always request packets from server's IP**
Costly as service provider has to pay for interconnection & bandwidth cost
Star topology
Applications: web, FTP, email, Telnet

Peer-to-peer:

Minimal or no reliance on dedicated servers
The communication is between **pairs of connected hosts**
Peers are often are not from service providers but **regular users**
Self-scalability, cost effective, no server infrastructure and server bandwidth
Generally **not secure** due to the highly distributed and open nature
The performance of the network depends on the **number of peers**
Applications: file sharing, peer assisted download, Internet Telephony

Remember, you are asked about architectural differences, not operational differences

R1.b (3Points) Describe the non persistent and persistent connection approach in HTTP. What are the pros and cons of the two approaches (+1point)?

Non-persistent:

Separate TCP connection for each request/response pair

Each TCP is closed after the server sends the object

TCP connection does not persist for other objects

Each TCP connection transport exactly one request message and one response message

Pros:

Able to create multiple parallel connections -> shortens the response time

Cons:

Brand new connection must be established and maintained for each requested object

-> places significant burden on the server

High memory usage (TCP buffer must be allocated, TCP variables must be kept)

Each object suffers a delivery delay of 2 RTTs

Persistent:

Same/single TCP connection for multiple request/response pairs

Servers leave the TCP connection open to serve subsequent requests and responses

Multiple objects can be transported in one connection

Only close connection after certain amount of time

Pros:

Save the number of connections, better use of one TCP connection

Pay the cost of setting connection only once for transporting multiple objects

-> less burden on the server

Can use pipelining to shorten response time

Reduced network congestion (because fewer connections)

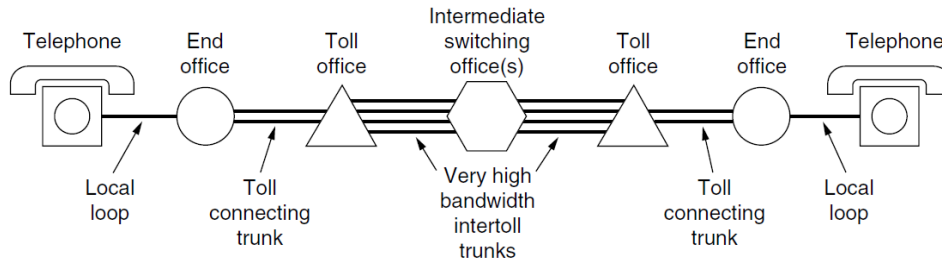
Lower memory usage (less TCP buffer and variables)

Cons:

Not fully utilized the connection (idle time) - this could be also a waste of (server) resources

Pros and cons are extra credits. You will get 1 pt if you discuss (correctly and meaningfully) pros and cons of BOTH approaches.

R1.c (5Points) Briefly describe the hierarchy of the public telephone network's architecture and its main components.



From the bottom of the hierarchy, we have these entities:

Telephones (Phones):

Users: homes, businesses

End offices (Switching offices):

Connect to phones via local loops - nearest to phones

Toll offices (Higher level switching offices):

Connect many switching offices via very high bandwidth trunks

-> These entities formed a national hierarchy of small redundancy - However, it's vulnerable if a few key toll offices become isolated

Local loops (last miles):

Analog twisted pairs wires, going to houses and businesses

Low capacity, weakest links, but provides everyone the access to the system

Trunks:

Digital fiber optic links

High capacity, able to handle many simultaneous calls

If you instead, describing the network in terms of LATA, LEX, IXC, IXC POP, tandem office: it is okay. However, remember that those terms only refer to offices component, you have to discuss local loops and trunks as well.

Review questions 2

R2.a (6Points) Compare TCP and UDP. What are the services offered to the application layer? Why is UDP more suited to multimedia streaming?

UDP:

Connectionless: send datagrams to IP without setting channels or datapaths - no handshake
Light weight: minimum protocol mechanism
Unreliable: the message can get lost without knowing
Not ordered: order of messages received is random
No congestion control: application layer will take care of congestion
Datagrams: individual packets are checked at receiver only
Unidirectional
Small packet header overhead: 8 bytes of overhead

TCP:

Connection-oriented: handshaking is a must
Heavy weight: 3 packets are needed to set up the connection
Reliable: guaranteed of service
Ordered: order of message is preserved, out-of-order data will be buffered
Congestion control: TCP handles congestion-control
Full duplex connections: two-way communication
Big packet header overhead: 20 bytes of overhead in every segment

UDP is more suited for multimedia streaming because:

UDP is stateless, suitable for large number of clients.

Transmission delay is less in UDP.

By design, UDP is unidirectional.

Streaming can tolerate small amount of packet loss, thus reliable data transfer is not absolutely critical.

Real-time applications react very poorly to TCP's congestion control

R2.b (7Points) Prove Little's theorem (no blocking).

Look at the proof in class or in the Appendix A

Notes: You have to explain all the notations you introduced in the proof. You can't just throw notations in your proof without explanations. Furthermore, explain the transitions between steps is critical in proving. This is not a long and complicated proof yet very important!

R2.c (7Points) Explain the difference between deterministic multiplexing and stochastic multiplexing. Does stochastic multiplexing better describe forwarding in circuit switched or packet switched networks? Why?

Stochastic multiplexing:

- Communication link is divided into arbitrary number of variable bit-rate digital channels or data stream
- Adapt to instantaneous traffic demand of data streams that are transferred over each channel
- Improve link utilization rate
- Packet-mode: packet oriented, better for packet switched network
- On demand service
- Randomized order of available slots
- Vary the delays
- Allow arbitrary number of division
- No wasted slots
- Link transmission capacity will be shared by only processes that have packets
- Carried out at datalink layer and above

Deterministic multiplexing: (e.g., FDM, TDM)

- Fixed divisions/sharing of the link
- Fixed order of available slots -> fixed delays
- Allocated resources can be wasted (silent periods) -> Less efficient -> More costly
- Shared by everyone in the link, regardless of activities
- Reserved resources regardless of demand (suitable for circuit switched network more)
- Reservation requires more complex, more costly implementation
- Carried out at physical layer

The key here is:

Deterministic multiplexing has a pre-determined fabric for resource sharing (TDM, FDM, space...makes no difference), so that one can predict when a packet from a given data stream will be served (which is connected with the performance guarantees of circuit switched networks).

In stochastic multiplexing, the amount and timing of allocated resource is random and function of the network conditions. Of course, this latter case suits packet switching, where you have packet buffering. The allocated transmission time is a function of the position in the buffer, which is, in the end, a random variable.

You have to explain explicitly why stochastic multiplexing better describes forwarding in packet switched networks. Comparing stochastic multiplexing and deterministic multiplexing alone will not get you full credit.

Review questions 3

R3.a (8Points) Define the phenomenon called **packet jitter**. Provide a qualitative description of the techniques used to **reduce jitter** in multimedia real-time applications.

Packet jitter:

Phenomenon that is due to **varying queuing delays** that a packet experiences in network's routers, the time from when a **package is generated at the source until it is received** can **fluctuate** from packet to packet.

In multimedia real-time applications, the receiver if ignore jitter and play audio chunks as soon as they are received, the resulting audio quality at the receiver is unintelligible.

Jitter can be removed using **sequence numbers** (self-explanatory), **time stamps** and a **playout delay**

Time stamp: Sender prepends each chunk with the time at which the **chunk was generated**

Playout delay: The assumption is **most packets will arrive before the playout time**. If packet arrives after the playout time, it is discarded and considered lost.

Fixed delay: Receiver plays out the chunk **exactly Q** (time unit) after the chunk is generated. So if a chunk is timestamped at the sender at time T, the receiver plays out the chunk at time T+Q. The choice of Q is the trade off between playback delay and packet loss.

Adaptive delay: **Estimate** the network delay and the **variance** of the network delay, then **adjust** the playout delay accordingly

Forward Error Correction (FEC) and Interleaving are not the techniques to reduce jitter. They are used to deal with packet loss, which is not the main thing asked here.

R3.b (12Points) Prove that in a M/M/1 system the probability that n users are in the system is

$$\pi(n) = (\lambda/\mu)^n \pi(0), \quad (10)$$

and that $\pi(0) = 1 - \lambda/\mu$ (infinite buffer case).

Look at the proof in class or in the Appendix A

Notes: p, P, ρ are easy to distinguish in printing but sometimes impossible on hand-written papers.

Thus, it is advisable to choose easy-to-distinguish symbols (on your final).

In this question, you must set up the probabilities of state transitions. Explanations of notations and between steps are required, that including show your works in solving equations. A lot of you missed these, thus lost points.

R3.c (8Points) Describe the leaky bucket mechanism. Why is this type of mechanism needed in priority queueing?

Leaky bucket mechanism: a traffic policing mechanism, to be implemented at the edge of the network to control the characteristics of the traffic injected. The leaky bucket enforces the traffic of a stream to stay within rate limits (controls injection rate (average rate, peak rate and burst size) into the network).

A leaky bucket can hold up to B tokens. Tokens are generated at rate R . If $R > B$ then the extra token is ignored, the bucket remains full of B tokens.

Each packet entering the network must have 1 token, otherwise it has to wait for an available token. The token is removed from the bucket

Because of the token-generation rate $R > B$, obviously the maximum burst size is B . This also makes the maximum long term average rate is R . Maximum number of packets that can enter the network for any interval T is $RT + B$

Priority queueing: packets must be classified according to explicit marking. Each priority class has its own queue. With this mechanism, we can choose packet to transmit first from the highest priority class. All other traffic can be handled when the highest priority queue is empty.

Leaky bucket mechanism is needed in priority queueing to avoid abuse of the prioritized traffic over the non-prioritized buffer. Otherwise, prioritized traffic can eat out all the bandwidth and leave nothing to the others.